

# Coin Change - Minimum Coins to Make Sum

Last Updated : 14 Mar, 2025



Given an array of **coins[]** of size **n** and a target value **sum**, where **coins[i]** represent the coins of different denominations. You have an **infinite supply** of each of the coins. The task is to find the **minimum** number of coins required to make the given value **sum**. If it is not possible to form the sum using the given coins, return **-1**.

## Examples:

**Input:** `coins[] = [25, 10, 5], sum = 30`

**Output:** 2

**Explanation :** Minimum 2 coins needed, 25 and 5

**Input:** `coins[] = [9, 6, 5, 1], sum = 19`

**Output:** 3

**Explanation:**  $19 = 9 + 9 + 1$

**Input:** `coins[] = [5, 1], sum = 0`

**Output:** 0

**Explanation:** For 0 sum, we do not need a coin

**Input:** `coins[] = [4, 6, 2], sum = 5`

**Output:** -1

**Explanation:** Not possible to make the given sum.

## Table of Content

- [\[Naive Approach \] Using Recursion –  \$O\(n^{\text{sum}}\)\$  Time and  \$O\(\text{sum}\)\$  Space](#)
- [\[Better Approach 1\] Using Top-Down DP \(Memoization\) -  \$O\(n \* \text{sum}\)\$  Time and  \$O\(n \* \text{sum}\)\$  Space](#)
- [\[Better Approach 2\] Using Bottom-Up DP \(Tabulation\) -  \$O\(n \* \text{sum}\)\$  Time and  \$O\(n \* \text{sum}\)\$  Space](#)
- [\[Expected Approach\] Using Space Optimized DP –  \$O\(n \* \text{sum}\)\$  Time and  \$O\(\text{sum}\)\$  Space](#)

**[Naive Approach ] Using Recursion –  $O(n^{\text{sum}})$  Time and  $O(\text{sum})$  Space**

This problem is a variation of the problem [Coin Change Problem](#). Here instead of finding the total number of possible solutions, we need to find the solution with the **minimum** number of coins.

The idea is to find the **minimum number of coins** required to reach the **target sum** by trying each coin denomination in the **coins[]** array. Starting from the target sum, for each **coin coins[i]**, we can either include it or exclude it. If we **include** it, we **subtract** its value from sum and recursively try to make the remaining amount with the same coin denominations. If we **exclude** it, we move to the next coin in the list.

Mathematically the recurrence relation will look like the following:

$$\text{minCoins}(i, \text{sum}, \text{coins}) = \min(1 + \text{minCoins}(i, \text{sum} - \text{coins}[i], \text{coins}), \text{minCoins}(i+1, \text{sum}, \text{coins}))$$

**Base cases:**

- $\text{minCoins}(i, \text{sum}, \text{coins}) = 0$ , if  $\text{sum} = 0$ .
- $\text{minCoins}(i, \text{sum}, \text{coins}) = \text{INTEGER MAX}$ , if  $\text{sum} < 0$  or  $i == \text{size of coins}$ .

C++   Java   Python   C#   JavaScript

```
// C++ program to find minimum of coins
// to make a given change sum
#include<bits/stdc++.h>
using namespace std;

int minCoinsRecur(int i, int sum, vector<int> &coins) {

    // base case
    if (sum == 0) return 0;
    if (sum < 0 || i == coins.size()) return INT_MAX;

    int take = INT_MAX;

    // take a coin only if its value
    // is greater than 0.
    if (coins[i]>0) {
        take = minCoinsRecur(i, sum-coins[i], coins);
        if (take != INT_MAX) take++;
    }
}
```

```

    }

    // not take the coin
    int noTake = minCoinsRecur(i+1, sum, coins);

    return min(take, noTake);
}

int minCoins(vector<int> &coins, int sum) {

    int res = minCoinsRecur(0, sum, coins);
    return res!=INT_MAX?res:-1;
}

int main() {
    vector<int> coins = {9, 6, 5, 1};
    int sum = 19;
    cout << minCoins(coins, sum);
    return 0;
}

```

## Output

3

## [Better Approach 1] Using Top-Down DP (Memoization) - $O(n \cdot \text{sum})$ Time and $O(n \cdot \text{sum})$ Space

If we notice carefully, we can observe that the above recursive solution holds the following two properties of [Dynamic Programming](#):

### 1. Optimal Substructure:

Minimum number of ways to make **sum** at index **i**, i.e., **minCoins(i, sum, coins)**, depends on the optimal solutions of the subproblems **minCoins(i, sum-coins[i], coins)**, and **minCoins(i+1, sum, coins)**. By comparing these optimal substructures, we can efficiently calculate the minimum number of coins to make target sum at index **i**.

### 2. Overlapping Subproblems:

While applying a recursive approach in this problem, we notice that certain subproblems are computed multiple times.

- There are only two parameters: **i** and **sum** that changes in the recursive solution. So we create a 2D matrix of size  $n \times (\text{sum} + 1)$  for [memoization](#).
- We initialize this matrix as -1 to indicate nothing is computed initially.
- Now we modify our recursive solution to first check if the value is -1, then only make recursive calls. This way, we avoid re-computations of the same subproblems.

C++

Java

Python

C#

JavaScript

```
// C++ program to find minimum of coins
// to make a given change sum
#include<bits/stdc++.h>
using namespace std;

int minCoinsRecur(int i, int sum, vector<int> &coins,
vector<vector<int>> &memo) {

    // base case
    if (sum == 0) return 0;
    if (sum < 0 || i == coins.size()) return INT_MAX;

    if (memo[i][sum] != -1) return memo[i][sum];

    int take = INT_MAX;

    // take a coin only if its value
    // is greater than 0.
    if (coins[i] > 0) {
        take = minCoinsRecur(i, sum - coins[i], coins, memo);
        if (take != INT_MAX) take++;
    }
    // not take the coins
    int noTake = minCoinsRecur(i + 1, sum, coins, memo);

    return memo[i][sum] = min(take, noTake);
}
```

```

int minCoins(vector<int> &coins, int sum) {
    vector<vector<int>> memo(coins.size(), vector<int>(sum+1,
-1));
    int res = minCoinsRecur(0, sum, coins, memo);
    return res!=INT_MAX?res:-1;
}

int main() {
    vector<int> coins = {9, 6, 5, 1};
    int sum = 19;
    cout << minCoins(coins, sum);
    return 0;
}

```

## Output

3

## [Better Approach 2] Using Bottom-Up DP (Tabulation) - $O(n \cdot \text{sum})$ Time and $O(n \cdot \text{sum})$ Space

The idea is to fill the **DP table** based on **previous** values. For each coin, we either **include** it or **exclude** it to compute the **minimum** number of coins needed for each sum. The table is filled in an iterative manner from  $i = n-1$  to  $i = 0$  and for each sum from 1 to sum.

The dynamic programming relation is as follows:

- if  $(\text{sum} - \text{coins}[i])$  is greater than 0, then  $\text{dp}[i][\text{sum}] = \min(1 + \text{dp}[i][\text{sum} - \text{coins}[i]], \text{dp}[i+1][\text{sum}])$
- else  $\text{dp}[i][\text{sum}] = \text{dp}[i+1][\text{sum}]$ .

C++

Java

Python

C#

JavaScript

```

// C++ program to find minimum of coins
// to make a given change sum
#include<bits/stdc++.h>
using namespace std;

```

```

int minCoins(vector<int> &coins, int sum) {
    vector<vector<int>> dp(coins.size(), vector<int>(sum+1, 0));

    for (int i=coins.size()-1; i>=0; i--) {
        for (int j=1; j<=sum; j++) {
            dp[i][j] = INT_MAX;
            int take = INT_MAX, noTake = INT_MAX;

            // If we take coins[i] coin
            if (j-coins[i]>=0) {
                take = dp[i][j-coins[i]];
                if (take != INT_MAX) take++;
            }

            if (i+1<coins.size())
                // not take the coin
                noTake = dp[i+1][j];

            dp[i][j] = min(take, noTake);
        }
    }

    if (dp[0][sum]!=INT_MAX) return dp[0][sum];
    return -1;
}

int main() {
    vector<int> coins = {9, 6, 5, 1};
    int sum = 19;
    cout << minCoins(coins, sum);
    return 0;
}

```

## Output

3

**[Expected Approach] Using Space Optimized DP –  $O(n \cdot \text{sum})$  Time and  $O(\text{sum})$  Space**

In **previous approach** of dynamic programming we have derive the relation between states as given below:

- if  $(\text{sum} - \text{coins}[i])$  is greater than 0, then  $\text{dp}[i][\text{sum}] = \min(1 + \text{dp}[i][\text{sum} - \text{coins}[i]], \text{dp}[i+1][\text{sum}])$
- else  $\text{dp}[i][\text{sum}] = \text{dp}[i+1][\text{sum}]$ .

If we observe that for calculating **current**  $\text{dp}[i][\text{sum}]$  state we only need previous row  $\text{dp}[i-1][\text{sum}]$  or current row  $\text{dp}[i][\text{sum} - \text{coins}[i]]$ . There is no need to store all the **previous** states just one previous state is used to compute result.

C++

Java

Python

C#

JavaScript

```
// C++ program to find minimum of coins
// to make a given change sum
#include<bits/stdc++.h>
using namespace std;

// Function to find the minimum number of
// coins needed to make the given sum
int minCoins(vector<int> &coins, int sum) {

    // Create a DP array to store the minimum coins
    // for each value up to sum
    vector<int> dp(sum+1, INT_MAX);

    // Base case: 0 coins are needed to make sum 0
    dp[0] = 0;

    // Iterate over each coin in the coins array
    for (int i=coins.size()-1; i>=0; i--) {

        // Iterate through all sums from 1 to the given sum
        for (int j=1; j<=sum; j++) {

            // Initialize take and noTake variables
            int take = INT_MAX, noTake = INT_MAX;

            // Check if we can take the current coin
            if (j-coins[i]>=0 && coins[i]>0) {
```

```

        // Get the minimum coins needed for
        // the remaining sum
        take = dp[j-coins[i]];

        // Increment take if it's a valid option
        if (take != INT_MAX) take++;
    }

    // If we have more coins available,
    // consider not taking the current coin

    if (i+1<coins.size())
    // Get the minimum coins needed without
    // taking the current coi
        noTake = dp[j];

    // Update the DP array with the minimum of
    // taking or not taking the coin
    dp[j] = min(take, noTake);
}
}

// found a solution, return the minimum
// coins needed, otherwise return -1
if (dp[sum]!=INT_MAX) return dp[sum];
return -1;
}

int main() {
    vector<int> coins = {9, 6, 5, 1};
    int sum = 19;
    cout << minCoins(coins, sum);
    return 0;
}

```

## Output

3

[Comment](#)**kartik**[+ Follow](#)

251

**Article Tags :**[Misc](#)[Dynamic Programming](#)[Mathematical](#)[DSA](#)[+9 More](#)

## Explore

[DSA Fundamentals](#)[Data Structures](#)[Algorithms](#)[Advanced](#)[Interview Preparation](#)[Practice Problem](#)

Do Not Sell or Share My Personal Information